
BUILDING R.O.B. - VOLUME 6

DUAL-ARM ROBOTICS

AMBER B1, URDF, CAN, UDP, LCM, and Ubuntu



AN EVIDENCE-BASED INTEGRATION AND COMMISSIONING GUIDE

Rodolfo Aramayo / Orbitus Robotics - First Maker Faire Edition

A model is not a safety certificate

Understand every frame before commanding motion.

This volume documents the two AMBER B1 arms from the checked-in Ubuntu snapshot, Python API, launch configuration, and robot-description files. It distinguishes directly observed formats from assumptions, identifies contradictions that must be resolved on the real machine, and begins every commissioning path with power removed or motion disabled.

The archived core executables are binaries, not complete source. This book can reproduce the observed runtime envelope, configuration, and client protocol; it cannot prove how the proprietary core implements every CAN transaction or rebuild those executables from source.

Copyright © 2026 Rodolfo Aramayo / Orbitus Robotics. Photographs are from the private ROB build archive unless otherwise noted. The generated frontispiece is original artwork derived from a ROB reference photograph. This independent educational project is not affiliated with or endorsed by any film studio or franchise owner. Product and company names remain the property of their respective owners.

First evidence-based draft: 2 August 2026. Build details marked **Builder input needed** are deliberate placeholders for the builder to complete.

POWERED ARMS CREATE CRUSH, PINCH, AND IMPACT HAZARDS

Use a qualified adult lead, a verified independent emergency stop, current and speed limits, physical clearance, controlled power, and a spotter. Perform discovery and status tests before enabling torque. Never stand inside either arm's swept volume, and never infer a safe limit from a URDF or example script.

The Building R.O.B. library

VOLUME 1	<i>Meet ROB: A Robot Is a Team of Systems</i> - ages 8-12
VOLUME 2	<i>Circuits and Signals with ROB</i> - ages 10-14
VOLUME 3	<i>Motion Workshop: Treads, Gears, and Fabrication</i> - ages 10-15
VOLUME 4	<i>Mission Control: Arduino, Mac, Networks, and Vision</i> - ages 12-16
VOLUME 5	<i>AI, Robotics, and Codex: Directing, Verifying, and Owning AI-Built Systems</i> - advanced makers
VOLUME 6	<i>Dual-Arm Robotics: AMBER B1, URDF, CAN, and Ubuntu</i> - advanced makers
MANUAL	<i>Building R.O.B.: The Complete Maker's Field Manual</i> - advanced builders

HOW TO USE THIS BOOK

Read Chapters 1-8 at a desk first. Build the manifest and run the read-only checks in Chapters 9-11 on the Ubuntu box. Only a responsible operator may authorize the staged motion tests.

ONE CURRENT ARDUINO, THREE HISTORICAL SKETCHES

The ROBArduino archive preserves Base, Torso, and Head sketches from an earlier three-Arduino design. The builder reports that present-day ROB uses **only the Base sketch**. This book therefore treats Base behavior as the current low-level firmware reference and labels Torso and Head behavior as retired historical evidence. Source files cannot prove which binary is flashed today; record the board identity, firmware hash, build environment, and upload date before operation.

TWO AMBER B1 ARMS, ONE DOCUMENTED CONTROL CHAIN

ROB carries two seven-joint AMBER B1 arms. The repository snapshot separates their Ubuntu-side cores as L-10 and R-11: network clients use UDP or LCM, each core binds a stable CAN interface, and CAN carries commands and feedback between that core and its arm actuators. The AmberHomeFolder tree is evidence from one Ubuntu machine, not a deployable image; never copy its credentials, logs, virtual environment, or machine-specific identifiers. Volume 6 documents the inspected protocol, URDF models, reproducible setup, verification gates, and unresolved configuration differences.

Contents

1	The arm system we actually have	1
1.1	An evidence ledger	1
2	Read the repository without copying a whole computer	2
2.1	Snapshot record	2
3	URDF is a geometric sentence	3
3.1	The single B1 chain	3
3.2	The dual model has a warning in its own source	4
4	Measure two arms into ROB's coordinate system	5
4.1	Mount-transform worksheet	5
5	CAN: stable identity before traffic	6
5.1	Prefer a deterministic host binding	6
5.2	CAN fault questions	6
6	Configure one core per side	7
7	The observed UDP protocol	8
7.1	Command catalogue	8
7.2	Gripper sequence	9
7.3	Protocol hardening needed	9
8	LCM and ROS 2 interfaces	10
9	Build a clean Ubuntu runtime	11
9.1	Reproduction procedure	11
9.2	Service order	12
10	A status-only Python lesson	13
11	Commission from no motion to bounded motion	14
11.1	Gate 0: paper review	14
11.2	Gate 1: power isolated	14
11.3	Gate 2: status only	14

11.4 Gate 3: supported, torque-limited joint identification	14
11.5 Gate 4: model agreement	14
11.6 Gate 5: independent single-arm envelopes	14
11.7 Gate 6: dual-arm coordination	15
12 Diagnose by layer	16
13 Connect the arms to Cerebro responsibly	17
14 What remains unknown	18
15 Reproduction acceptance sheet	19
16 Teach it as a systems lesson	20

The arm system we actually have

ROB's manipulation system is not one abstract "robot arm." It is two AMBER B1 arms, each with seven revolute joints and a gripper actuator. The inspected Ubuntu snapshot names the deployed sides L-10 and R-11. Their current launch files separate the sides at every important boundary:

Concern	Left arm	Right arm
Ubuntu directory	amber/L-10	amber/R-11
CAN interface	can10	can11
LCM prefix	Left_	Right_
UDP port	26001	26002
end-effector link	Lseven_Link	Rseven_Link
gripper actuator number	8	8

The repository supports this architecture diagram:

```

1 operator / Cerebro / test client
2   |
3   UDP or multicast LCM
4   |
5   Ubuntu AMBER core processes
6   /      \
7 L-10 core :26001  R-11 core :26002
8   | can10        | can11
9   |              |
10 seven left joints  seven right joints
11 + gripper #8      + gripper #8

```

This distinction matters. The Python API does **not** place CAN frames on the wire. It sends packed UDP datagrams to an AMBER core. The core owns the CAN-side exchange with the actuators. The saved LCM types provide a higher-rate publish/subscribe route into that same core. A future replacement core would need a separately verified actuator-level CAN specification; that specification is not present in the inspected source tree.

An evidence ledger

Use four labels throughout commissioning:

- **Observed:** directly readable in a source file, launch file, URDF, or captured configuration.
- **Derived:** calculated from observed values, such as converting radians to degrees.
- **Reported:** supplied by the builder but not independently demonstrated by the snapshot.
- **Unknown:** requires measurement, vendor documentation, or a controlled test.

The two arms, seven-joint count, ports, prefixes, core settings, UDP structures, and URDF geometry are observed. The exact mounting transforms on ROB, current safe payload, braking behavior, CAN adapter bitrate interpretation, and the firmware actually running today still require confirmation.

Read the repository without copying a whole computer

The arm evidence is distributed across three project locations:

```

1 AmberHomeFolder/
2   amber/
3     L-10/launch.json           left runtime configuration
4     R-11/launch.json           right runtime configuration
5     amber_core/urdf/           single- and dual-arm models
6     amber_core/init/initCan.sh adapter discovery and CAN binding
7     sin_wave/rawLcm/*.lcm      LCM message definitions
8     Install/                   historical dependency scripts
9
10  Amber URDF/amber_b1.urdf      independent single-arm model
11
12  Cerebro/Amber-PythonAPI/
13    Amber V2 API/amber_api/     UDP client implementation
14    Amber v1 API/               older packet examples

```

AmberHomeFolder is a captured home directory. It also contains histories, logs, cached packages, a virtual environment, device identifiers, and SSH material. Those files are useful forensic evidence but are not a golden image. Do not publish them, synchronize their credentials, or recursively copy them onto a replacement Ubuntu box. Build a clean host from a reviewed manifest and copy only the authorized runtime artifacts, URDFs, message definitions, and configuration.

The `amber_core_L` and `amber_core_R` programs are compiled executables. Because corresponding complete source and a reproducible build recipe are absent, record their architecture, cryptographic digest, owner, permissions, and provenance before use. Reproducing the environment is not the same as reproducing the binary.

Snapshot record

For each deployment, record:

```

1 host name and Ubuntu release:
2 machine architecture:
3 kernel:
4 AMBER core source/provenance:
5 left core SHA-256:
6 right core SHA-256:
7 left launch SHA-256:
8 right launch SHA-256:
9 URDF SHA-256:
10 Python API SHA-256:
11 CAN adapter label -> stable interface:
12 tested hardware revision:
13 operator and date:

```

Keep this manifest beside the deployment package, not inside an unfiltered home-folder archive.

URDF is a geometric sentence

A Unified Robot Description Format file is XML that describes a tree of **links** connected by **joints**. Links carry visual geometry, collision geometry, and optionally mass and inertia. Joints name a parent link, child link, origin transform, axis, type, and limits. Together they let visualization, forward kinematics, inverse kinematics, motion planning, and collision tools speak about the same mechanism.

A URDF answers “where would the next link be if this joint had a known angle?” It does not, by itself, answer:

- where the physical arm is mounted on ROB;
- whether an encoder zero agrees with the model zero;
- whether a limit is mechanically safe under load;
- whether collision meshes include cables, grippers, ROB’s body, or a carried object;
- how quickly the real controller stops after communication loss;
- whether the physical robot matches the exported CAD revision.

URDF coordinates use SI conventions: meters, radians, kilograms, and seconds-derived units. An **origin** contains xyz translation and roll-pitch-yaw rotation. A revolute joint’s angle rotates its child about the declared axis. Composition order and frame ownership must remain explicit; changing signs until a picture “looks right” creates hidden calibration debt.

The single B1 chain

The independent Amber URDF/amber_b1.urdf defines base_link, seven numbered links, and joints joint1 through joint7. Every joint axis is the local positive Z axis. These are model limits, not permission to drive to an endpoint:

Joint	origin xyz (m)	origin rpy (rad)	lower / upper (rad)	modeled velocity
1	0 0 0.0825	0 0 0	-2.4435 / 2.4435	3.1415
2	0 0 0.0853	-1.5708 0 0	-2.3213 / 2.3213	3.1415
3	0 -0.1289 0	1.5708 0 0	-2.2863 / 2.2863	3.1415
4	0 0 0.0853	1.5708 0 0	-2.2863 / 2.2863	3.1415
5	0 0.1251 0	-1.5708 0 0	-2.2863 / 2.2863	5.23
6	0 0 0.0891	-1.5708 0 0	-2.2863 / 2.2863	5.23
7	0 -0.1591 0	1.5708 0 0	-3.05 / 3.05	5.23

The V2 Python wrapper applies a generic software range of approximately ± 2.0944 radians to all seven joints. That is not identical to the joint-specific URDF limits. Treat both as inputs to a reviewed safety envelope, never select whichever value permits more motion, and add a robot-specific conservative limit layer above the vendor client.

The dual model has a warning in its own source

`DualArm.urdf` contains both L... and R... chains attached to a shared `base_link`. Its leading comment says axes 2, 4, and 6 have opposite rotation directions at the driver layer and warns that left/right labels were defined from an observer-facing convention. The launch files likewise give the solver an alternating rotation-direction vector:

```
1 [1, -1, 1, -1, 1, -1, 1]
```

This is not decorative metadata. A sign error can send a physical joint opposite the planned direction. Create one canonical mapping table with columns for physical side, software side, CAN interface, network port, LCM prefix, URDF chain, joint sign, encoder zero, and photographed hardware label. Verify one joint at a time with torque limited and the arm supported.

The current right launch file uses `DualArm.urdf`, while an alternate right configuration uses `DualArmR.urdf`. Reconcile that difference before commissioning. Do not assume the newest filename is correct merely because it is more specific.

Measure two arms into ROB's coordinate system

The dual-arm CAD base is not automatically ROB's torso frame. Measure the rigid transform from a named ROB body frame to each arm's `base_link`. A useful frame tree is:

```

1 rob_base
2   -> torso_yaw
3     -> left_arm_mount -> left base_link -> Lseven_Link -> left_tool
4     -> right_arm_mount -> right base_link -> Rseven_Link -> right_tool
5     -> camera_mount -> camera_optical

```

Define axes before measuring: for example, X forward, Y left, Z up. Put a durable datum on the torso and each arm plate. Measure translation with calipers, a height gauge, a jig, or a surveyed target; measure rotation from machined surfaces or a calibrated fixture. Take repeated measurements and record uncertainty rather than adding false decimal places.

Mount-transform worksheet

For each side record:

Field	Left	Right
physical label / serial		
torso datum description		
translation x, y, z (m)		
roll, pitch, yaw (rad)		
method and instrument		
repeated-measurement spread		
tool-center-point offset		
cable/gripper collision additions		
reviewer and date		

Validate the transform with several poses, not only a home pose. Compare predicted joint centers and tool pose against measured or vision-estimated landmarks. A transform that fits one image may conceal a scale, sign, time-alignment, or lens-calibration error.

CAN: stable identity before traffic

Linux may assign `/dev/ttyACM0`, `/dev/ttyACM1`, and later numbers according to discovery order. Moving a USB adapter to another hub can reorder them. The archived improved initialization script addresses this by reading each adapter's USB serial number, looking it up in `SerialNumber.txt`, and binding the matching adapter to a stable SocketCAN name such as `can10` or `can11`.

The script then runs `slcand` with `-o -c -s8`, brings the interface up, and sets transmit queue length to 1000. Preserve the exact adapter option as an observed configuration value; verify its bitrate meaning against the installed adapter and `slcand` version before connecting hardware.

Two archived `SerialNumber.txt` copies disagree. Unique serials are local inventory, not book content. Read the labels or query the actual adapters, create one authoritative mapping, protect it as machine configuration, and remove obsolete copies.

Prefer a deterministic host binding

A production setup should avoid parsing historical `dmesg` text and writing temporary files in the working directory. Use `udev` rules or a small system service that reads `/dev/serial/by-id`, validates that exactly one adapter maps to each logical side, refuses duplicates or unknown devices, creates the intended SocketCAN interface, and verifies it before starting a core.

Read-only checks include:

```
1 ls -l /dev/serial/by-id/
2 ip -details link show can10
3 ip -details link show can11
4 ip -statistics link show can10
5 ip -statistics link show can11
```

Do not transmit diagnostic frames from a generic CAN utility unless the actuator-level protocol and safe state are known. `candump` can observe traffic, but captures may contain device identifiers or operational data and must be handled accordingly.

CAN fault questions

For each side test and record: adapter absent at boot, swapped adapters, duplicate serial mapping, bus disconnected, bus-off recovery, core crash, client timeout, stale command, Ubuntu restart, and E-stop. The safe response must be measured on the real system. A quiet bus is not proof that an energized arm will hold, brake, or become harmless.

Configure one core per side

Both current launch files select `CAN_BUS`, seven degrees of freedom, a 200 Hz control frequency, gear ratios of 50, a gripper numbered 8, Drake solving, UDP, and ROS 2. Their maximum-velocity, acceleration, and jerk arrays are controller configuration units. The files do not prove those values are safe physical SI limits.

Important left settings are:

```
1 Hub_CAN_Name: can10
2 LCM Prefix: Left_
3 URDF_Path: ./urdf/dual_b1/DualArmL.urdf
4 EEF_Name: Lseven_Link
5 UDP_Port: 26001
```

Important right settings are:

```
1 Hub_CAN_Name: can11
2 LCM Prefix: Right_
3 URDF_Path: ./urdf/dual_b1/DualArm.urdf
4 EEF_Name: Rseven_Link
5 UDP_Port: 26002
```

Keep separate working directories if the executable resolves the URDF path relative to its current directory. Capture standard output and standard error under a supervised service. Give each core its own non-root account where possible, explicit working directory, read-only configuration, restart policy, resource limits, and dependency on its CAN interface. Do not automatically restart into an energized motion mode.

The observed UDP protocol

The V2 Python API uses packed ctypes. Structure records sent in UDP datagrams. Every observed message begins with:

Field	Type	Meaning
cmd_no	unsigned 16-bit	operation number
length	unsigned 16-bit	structure byte length
counter	unsigned 32-bit	request/reply correlation value

The structures set `_pack_ = 1`, so there is no padding. They do not declare explicit network byte order; current serialization therefore relies on the client's native representation. That is an implementation fact, not a good cross-platform protocol guarantee.

Command catalogue

- **1 – status:** no request payload. The reply carries eight joint positions, eight joint speeds, six Cartesian positions, six Cartesian speeds, and arm angle.
- **4 – joint move:** eight floats plus a duration float. The reply carries a one-byte response.
- **6 – Cartesian move:** XYZ, RPY, arm angle, and duration as floats. The reply carries a one-byte response.
- **7 – calibration:** an unsigned 32-bit actuator number. The reply carries a one-byte response.
- **9 – gripper:** unsigned 16-bit action, unsigned 16-bit intensity, and Boolean version. The reply carries a one-byte response.
- **10 – set mode:** an unsigned 16-bit mode. The reply carries a one-byte response.
- **110 – get mode:** no request payload. The reply carries seven unsigned 16-bit mode values.

The high-level wrapper exposes seven joint targets even though the status and move packet allocate eight floats. Gripper control is separate and uses actuator number 8. Preserve this difference when writing another client.

Observed modes are inactive 0, active 1, position 2, speed 3, and current 4. The API documentation says transitions involving position or current pass through active mode and may briefly cut power, allowing an unsupported arm to drop. Support the mechanism before mode changes and verify actual behavior with the vendor and a controlled test.

The API uses meters and radians at its high-level interface. `move_j` accepts seven joint angles and a duration. `move_c` accepts XYZ plus roll-pitch-yaw and a duration. `get_status` returns the first seven joint positions and a six-element Cartesian pose. "Position returned" does not prove the physical pose is correctly zeroed after a controller boot.

Gripper sequence

The V2 examples calibrate actuator 8 after each power cycle, then use action 0 for open and action 1 for close with an integer intensity. Calibration moves hardware: keep fingers, cables, and objects clear. The code alone does not establish a safe force or intensity for ROB's installed grippers.

Protocol hardening needed

The current client waits up to three seconds and accepts the next datagram. It does not consistently validate source endpoint, command number, declared length, counter, exact received size, freshness, or authentication before decoding. UDP also offers no delivery, ordering, or uniqueness guarantee.

A production adapter should:

1. bind each arm to an allowlisted endpoint and side identity;
2. encode endianness explicitly or preserve a tested compatibility codec;
3. validate datagram size before deserialization;
4. match command and counter to an outstanding request;
5. reject stale, duplicate, unsolicited, or non-finite values;
6. enforce conservative joint, rate, workspace, and mode policy locally;
7. use an authenticated protected network or authenticated gateway;
8. stop issuing motion on timeout without claiming that a software stop replaces the E-stop;
9. log decisions without logging credentials;
10. fuzz the decoder away from hardware.

The older V1 examples commonly use port 25001, whereas the current two-core launch files use 26001 and 26002. Select ports from reviewed configuration, not from whichever example happens to run.

LCM and ROS 2 interfaces

The saved LCM configuration composes channels from a side prefix and suffix. Examples are `Left_PosCmd`, `Left_ArmStatus`, `Right_PosCmd`, and `Right_ArmStatus`. Other configured suffixes include position plans and tasks, inverse- and forward-kinematics tasks, solver responses, mode changes, and gripper control.

The raw definitions establish these payloads:

LCM type	fields
<code>posCmd_t</code>	seven double joint targets
<code>rosData_t</code> / generated arm status	seven positions, velocities, currents, and statuses
<code>cartesian_t</code>	six XYZ/RPY doubles and duration
<code>gripperCtrl_t</code>	8-bit action and 32-bit intensity
<code>simplePositionTask_t</code>	timestamp, actuator count, one point, duration
<code>positionTask_t</code>	timestamp, actuator count, point sequence, time sequence
<code>positionPlan_t</code>	frequency, timestamp, actuator count, point sequence
<code>solverRespond_t</code>	32-bit response

The archived examples use multicast at `239.255.76.67:7667` with TTL 10 and add a multicast route. Multicast can reach more listeners than expected. Put it on a controlled robotics network, choose the intended interface explicitly, and firewall unrelated hosts. Generate language bindings from the checked-in `.lcm` files so the schema hash remains consistent; do not hand-maintain parallel definitions.

The launch files also enable ROS 2 and historical install scripts include Humble, MoveIt, controllers, `xacro`, and joint-state tools. The snapshot does not show a complete reviewed ROS 2 launch package for ROB. Treat ROS 2 capability as enabled configuration, not proof that names, quality-of-service policy, controller ownership, and safety behavior are production-ready.

Build a clean Ubuntu runtime

Begin with a supported Ubuntu release compatible with the authorized AMBER core binary and required Drake/ROS packages. The historical scripts target ROS 2 Humble and install CMake, `net-tools`, `can-utils`, Python 3 compatibility, `gflags`, `glog`, Drake, MoveIt, and controller packages. They are evidence, not a safe unattended installer: one script pipes a remote GitHub script directly into a shell, and package repositories and keys can change.

Reproduction procedure

1. Record the original host manifest and binary hashes.
2. Install a fresh minimal Ubuntu system; patch it before connecting ROB hardware.
3. Create a dedicated unprivileged runtime account. Keep SSH keys outside the deployment archive.
4. Install reviewed, pinned packages from official repositories. Save package versions and repository fingerprints.
5. Install `can-utils`, the adapter's `slcand` support, required C/C++ runtime libraries, and only the ROS/Drake components actually used.
6. Copy authorized core binaries, launch JSON, URDF meshes, and LCM definitions into versioned application directories. Do not copy `.ssh`, shell history, logs, caches, or the archived virtual environment.
7. Create and verify the stable adapter-to-can10/can11 mapping.
8. Validate JSON syntax, URDF XML, mesh paths, file permissions, ports, prefixes, side labels, and hashes while arm power is isolated.
9. Start each core under a service supervisor with its own working directory and logs.
10. Run network status probes before requesting any control mode.
11. Save the completed acceptance record and rollback package.

Useful non-motion checks are:

```

1  uname -a
2  lsb_release -a
3  ip -br link
4  ss -ltnp
5  sha256sum amber_core_L amber_core_R
6  xmllint --noout path/to/model.urdf
7  python3 -m compileall path/to/reviewed/client

```

Run `ldd` on the core executables and record every resolved library, but do not assume “not found” can be fixed by copying random libraries from the old host. Resolve dependencies from known packages or the authorized vendor distribution.

Service order

The safe logical order is network policy, CAN discovery, CAN validation, left core, right core, read-only status clients, operator UI, then a separately authorized mode transition. Shutdown reverses authority first: stop new commands, return through the documented mode procedure, verify the result, stop clients and cores, then isolate energy according to the hardware procedure.

A status-only Python lesson

Use the repository's reviewed V2 package rather than reproducing packet layouts in multiple applications. The first test should only request state:

```
1 from amber_api import Amber_Robot
2
3 left = Amber_Robot("LEFT_CORE_ADDRESS", 26001, joint_count=7)
4 right = Amber_Robot("RIGHT_CORE_ADDRESS", 26002, joint_count=7)
5
6 for name, arm in (("left", left), ("right", right)):
7     joint_position, cartesian_pose = arm.get_status()
8     print(name, joint_position, cartesian_pose)
```

Run this on an isolated, authorized robotics network with motion disabled. Confirm seven finite values, plausible ranges, correct side identity, stable repeated readings, and behavior when one core is intentionally unavailable. Do not call `set_mode`, `move_j`, `move_c`, `zero`, `calibration`, or `gripper` functions during the discovery test.

The repository wrapper has details to audit before production. Its default wait tolerances differ between layers, and one Cartesian wait default is represented differently from the code that indexes it. Add tests for defaults, short replies, wrong counters, NaN, infinity, lost packets, reordering, and simultaneous left/right requests.

Commission from no motion to bounded motion

Gate 0: paper review

Confirm ownership of the stop system, swept-volume boundary, side mapping, power isolation, support fixture, conservative limits, rollback, and observer roles. Resolve the right-URDF discrepancy and serial-map discrepancy.

Gate 1: power isolated

Validate host, files, hashes, adapters, interfaces, ports, process permissions, URDF parsing, and network policy. Confirm the physical E-stop interrupts the intended energy path using the approved electrical test procedure.

Gate 2: status only

Start one core and one read-only client. Verify source endpoint, side, joint count, finite values, update timing, disconnection behavior, and logs. Repeat independently for the second side. Then observe both without commanding either.

Gate 3: supported, torque-limited joint identification

With the arm mechanically supported and the area clear, an authorized operator enables the minimum suitable mode and requests a tiny change to one joint. A second person watches the physical joint and stop. Record commanded sign, observed sign, encoder change, current, latency, and stop result. Return to the approved neutral state before advancing to the next joint.

Gate 4: model agreement

Compare measured link landmarks with forward-kinematics predictions over several conservative poses. Validate left/right frames, joints 2/4/6 signs, tool transforms, and torso mounts. Do not run inverse kinematics until forward kinematics and feedback agree.

Gate 5: independent single-arm envelopes

Test each arm alone inside a conservative workspace that excludes ROB, the other arm, cables, observers, and floor. Measure stopping and fault behavior. Add collision objects for the torso, head, shoulder structure, gripper, and cable volume.

Gate 6: dual-arm coordination

Only after both single-arm cases pass should a planner own both arms in one collision model. Test mirrored commands, crossing workspaces, communication loss on one side, cancellation, and one-arm faults. "Two scripts happened to run" is not coordinated dual-arm control.

Diagnose by layer

- **No can10 or can11:** inspect adapter identity, udev/service state, slcand, and link state. Do not yet conclude that arm electronics failed.
- **Core will not start:** inspect working directory, launch JSON, URDF and mesh paths, and libraries. Do not yet conclude that CAN is defective.
- **UDP timeout:** inspect the process, listener, address, port, firewall, and route. Do not yet conclude that the arm is powered off.
- **Wrong arm responds:** inspect endpoint, port, prefix, CAN mapping, and physical label. Do not yet conclude that joint signs are correct.
- **Model bends backward:** inspect side convention, joints 2/4/6 mapping, zero, and URDF selection. Do not yet conclude that hardware must be rewired.
- **Status jumps at boot:** inspect zeroing state, mode transition, and packet validation. Do not yet conclude that visual pose is trustworthy.
- **LCM silence:** inspect multicast interface, route, channel prefix, and schema version. Do not yet conclude that UDP is also broken.
- **Gripper does nothing:** inspect actuator 8 identity, required calibration, mode, and response validation. Do not respond by assuming higher intensity is safe.

Collect one synchronized incident bundle: UTC time, host manifest, core hashes, launch hashes, interface statistics, process status, sanitized logs, request counter, response source, operator action, and physical observation. Never collect private keys or passwords into a diagnostic archive.

Connect the arms to Cerebro responsibly

Cerebro should expose one typed arm service rather than launching miscellaneous Python scripts with implicit addresses. Its interface should name side, joint, units, duration, correlation identifier, desired mode, deadline, and operator authority. Returned state should include measured joint position, velocity, current/status when available, receive time, source, mode, and confidence in calibration.

Keep four layers separate:

1. **Transport adapter:** validates UDP/LCM and reports communication health.
2. **State estimator:** time-aligns joint feedback, camera observations, and ROB transforms.
3. **Safety policy:** enforces state, limits, workspace, collision, freshness, and authority.
4. **Behavior layer:** asks for a grasp, gesture, or pose without gaining raw packet access.

Visual pose estimation can detect disagreement and help calibrate mounting or joint offsets, but it should not overwrite joint truth continuously without observability checks. Some poses hide joints; cameras occlude; symmetric links create ambiguous solutions; rolling shutter and latency distort motion. Fuse visual evidence with actuator feedback and keep uncertainty explicit.

For every command, the UI should show the selected physical arm, current mode, last feedback age, calibration revision, joint values, network/CAN health, and whether motion authority is present. A green connection indicator must never mean “safe to enter the workspace.”

What remains unknown

The repository gives ROB a strong starting point, but honest documentation preserves these open items:

- authorized source and version history for the Ubuntu core binaries;
- exact actuator-level CAN identifiers, frame encodings, checksums, timing, and fault semantics;
- the adapter option's verified bitrate on the installed hardware;
- authoritative stable serial-to-side inventory;
- correct right-arm URDF selection;
- exact measured torso-to-arm and arm-to-tool transforms;
- encoder-zero behavior across power cycles;
- joint, current, temperature, payload, and gripper limits for ROB's installation;
- collision geometry including cables and body additions;
- core behavior on stale command, malformed packet, bus-off, client loss, and E-stop;
- authenticated network design and software-update provenance;
- complete reproducible source build for the AMBER core.

Unknown does not mean failure. It means the next experiment has a clear question.

Reproduction acceptance sheet

Before labeling a replacement Ubuntu box ready, require signatures for:

- ☐ clean operating-system install and patch record;
- ☐ pinned package and repository manifest;
- ☐ authorized binary provenance and matching hashes;
- ☐ no credentials, history, caches, or old logs copied from the snapshot;
- ☐ stable physical left/right adapter mapping;
- ☐ can10 and can11 independently verified;
- ☐ launch JSON and URDF syntax verified;
- ☐ right-arm URDF choice resolved;
- ☐ unique UDP ports and LCM prefixes verified;
- ☐ read-only status from each side tested alone and together;
- ☐ packet source, length, command, counter, freshness, and value validation tested;
- ☐ physical E-stop and fault behavior tested by the responsible engineer;
- ☐ conservative joint and workspace policy installed;
- ☐ mounting and tool transforms measured with uncertainty;
- ☐ forward kinematics checked against multiple physical poses;
- ☐ single-arm tests passed before dual-arm tests;
- ☐ sanitized logs, rollback package, and operator procedure archived.

Teach it as a systems lesson

The most important AMBER lesson is not a magic command. It is traceability across representations. A child may see a hand wave. An engineer must be able to follow that event backward: tool pose, seven joint angles, a planned trajectory, a side-specific channel, a validated network message, one Ubuntu core, one stable CAN interface, actuator feedback, and a physical mechanism attached to a measured frame.

Build a paper exercise with two cardboard seven-link chains. Label every joint and give each arm a different color. Learners route a “move left wrist” card through behavior, safety policy, network, Ubuntu core, CAN, joint, and feedback stations. Insert fault cards: swapped adapter, stale packet, wrong sign, hidden camera marker, blocked arm, or missing status. The team wins by refusing unsafe ambiguity and locating the layer that owns the evidence—not by moving fastest.

That is how two complex arms become understandable: not by hiding the details, but by giving every detail a name, a source, a test, and a responsible human owner.